

Chapitre I : Introduction du langage JAVA

1) Présentation de JAVA :

1.1 Un peu d'historique :

- Java né en 1991 par James Gosling dans une équipe de Sun Microsystem.
- Le but était d'étudier l'impact de la convergence entre les systèmes embarqués et les ordinateurs.
- En 1993, l'émergence de l'internet a donné au langage sa chance, car il permettait d'exécuter des programmes dans des pages HTML **indépendamment de la plateforme**.

1.2 Caractéristiques de JAVA :

- Langage orienté objet,
- Syntaxe très proche du C,
- Portable/Multiplateforme puis qu'il est interprété : grâce à l'exécution par une machine virtuelle JVM « ***write once, run everywhere*** »,
- Programmation concurrente (multi-tâches),
- Sûr (il est fortement typé et plusieurs vérifications sont effectuées au chargement des classes et durant leur exécution),

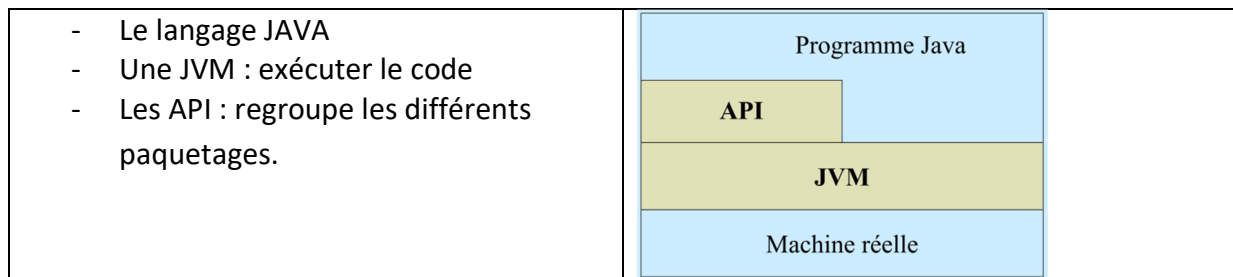
1.3 Les éditions :

Il existe trois éditions (environnement) différentes pour JAVA :

- JAVA SE (Standard Edition) pour desktop,
- JAVA EE (Entreprise Edition) pour serveur
 - o Servlet, JSP, JSF
 - o JTA/JTS, EJB
 - o JMS
 - o JavaMail
- JAVA ME (Mobile Edition) pour PDA, Smartphone (JAVA SE moins certains paquetages).

1.3) Spécification de JAVA :

Java, c'est en fait :



- Java n'est pas normalisé ; son évolution est gérée par le JCP (Java CommunityProcess ; <http://www.jcp.org/>) dans lequel Oracle tient une place prépondérante

2) Premier Programme JAVA :

Code source :

- JAVA est purement OO : tout programme a au moins une classe.

```
FichierSalem.java
public class Salem.java {
public static void main (String[]args) {
System.out.println("Salem");
}
}
```

- La classe Salem est public, donc le fichier qui la contient doit s'appeler (en tenant compte des majuscules et minuscules) Salem.java.

Compilation :

- Oracle fournit le compilateur javac avec le JDK.
- **javac Salem.java** crée un fichier «**Salem.class**» qui contient le bytecode, situé dans le même répertoire que le fichier «**.java**»

Exécution :

- Oracle fournit le programme java qui simule une JVM.
- **Java Salem** (nom de la classe, pas de suffixe **.class**) exécute le bytecode de la méthode main de ma classe Salem.

Architecture d'une application JAVA :

- Une application JAVA est un ensemble de fichiers « .java »,
- Chaque fichier « .java » contient **une ou plusieurs** classes,
- **Au plus** une définition de classe public par fichier « .java » (avec nom du fichier = nom de la classe publique).

Exemple :

```
/** Modélise un point de coordonnées x, y */
public class Point
{
    private int x, y;
    public Point(int x1, int y1) {
        x = x1; y = y1;
    }
    public double distance(Point p) {
        return Math.sqrt((x-p.x)*(x-p.x) + (y-p.y)*(y-p.y));
    }
}
/** Teste la classe Point */
class TestPoint
{
    public static void main(String[] args) {
        Point p1 = new Point(1, 2);
        Point p2 = new Point(5, 1);
        System.out.println("Distance : " + p1.distance(p2));
    }
}
```

- La compilation du fichier Point.java fournit 2 fichiers classes : **Point.class** et **TestPoint.class**
- On lance l'exécution de la classe **TestPoint** qui a la méthode main : **java TestPoint**

3) Les éléments de base :

3.1 Les identificateurs :

Un identificateur JAVA :

- Est de longueur quelconque,
- Commence par une lettre Unicode (caractère ASCII recommandés),
- Peut ensuite contenir des lettres ou des chiffres ou le caractère souligné « _ »,
- Ne doit pas être un mot-clé ou les constantes true, false ou null.

Conventions :

- Les noms de classes commencent par une majuscule (ce sont les seuls avec les constantes) : **Cercle, Object**
- Les mots contenus dans un identificateur commencent par une majuscule : **UneClasse, uneMethode, uneAutreVariable**,
- Les constantes sont en majuscules avec les mots séparés par le caractère souligné « _ » : **UNE_CONSTANTE**
- Si possible, des noms pour les classes et des verbes pour les méthodes.

3.2 Les commentaires :

Il y a 3 manières pour ajouter de commentaire à votre code :

Sur une seule ligne :	//voici un commentaire
Sur plusieurs lignes :	/* première ligne de commentaire suite de commentaire */
Documentation automatique par javadoc	/** * Cette méthode calcule ... * Elle utilise ... */

3.3 Les types de base :

Type de base	Type	Nombre de bits	Valeurs possibles	Valeurs par défaut
boolean	Booléen		true et false	false
byte	Entier	8	signé	0
short	Entier	16	signé	0
int	Entier	32	signé	0
long	Entier	64	signé	0
float	Virgule flottante	32	IEEE 754	0
double	Virgule flottante	64	IEEE 754	0
char	Caractère	16	Unicode	\u0000

Remarque : Il faut suffixer les constantes float par **f**, et les constantes long par **l**.

Exemple : `float = 12.3f ;`

3.4 Transtypage/casts :

Parfois, il est nécessaire de forcer le programme à considérer une expression comme étant d'un type différent de son type réel/déclaré. On utilise pour le **Cast/transtypage**.

En Java, 3 cas sont autorisés pour les casts :

- entre types primitifs,
- entre classes mère/ancêtre et classes filles, (détaillé plus loin)
- entre type primitifs et objets (détaillé plus loin)

Conversion entre type primitifs :

On distingue deux types de conversion :

- **Conversion implicite** : d'un type petit vers un type grand.
- **Conversion explicite** : d'un type grand vers un type petit, cette conversion peut engendrer perte de données, pour cela JAVA oblige le programmeur de demander la conversion explicitement :

Transtypage : nombreTypeForcé = (TypeForcé)expression

Exemple :

Affectation statique	<code>short s = 1000000000 ; //erreur</code>
Cast explicite	<code>int i = 60; short b = (short) (i + 5);</code>
Flottant vers entier	<code>int i = (int)1.99; // i = 1, et pas 2</code>
	<code>byte b1 = (byte)20; byte b2 = (byte)15; byte b3 = b1 + b2; provoque une erreur (b1 + b2 est un int) La dernière ligne doit s'écrire <code>byte b3 = (byte) (b1 + b2);</code></code>

Cast entre entiers et caractère :

Ils font correspondre un entier et un caractère qui a comme code Unicode la valeur de l'entier.

<code>char → int, long</code>	Cast implicite
<code>char → short, byte</code>	Explicite, char peut aller de 0 à 65.535, et short ne va que jusqu'à 32.767
<code>long, int, short, byte → char</code>	Les entiers sont signés et pas le char, donc c'est explicite.